FutureProof Financial

Platform Strategy & Build Brief

Full technical build specification

The build specification for the platform that turns one EPM product into one engine many licensed lenders run on, across markets. Architecture, domain model, four workstreams in build-level detail, the customer journeys, the build/test/deploy/operate process, security and compliance, and a phased roadmap with acceptance criteria — written so a developer (with AI coding agents) can start cold.

One engine. Every market. Run by agents.

Internal — for the team · June 2026

0. Read me first

What this is. The build specification for the platform that turns one EPM product into one engine that many licensed lenders run on, in many markets. It covers the architecture, the domain model, four workstreams in build-level detail, the customer journeys, the build/test/deploy/operate process, security and compliance, and a phased roadmap with acceptance criteria.

Who it's for. The engineer who builds the platform, working with AI coding agents (e.g. Claude Code). Assumes fluency in Rails and PostgreSQL. Where a decision has been made, it is stated and justified; where one is open, it is flagged in Section 16.

How to use it. Build in the order of Section 15 (the roadmap). For each component, follow the loop in Section 10 and build to the acceptance criteria in its workstream. Treat Sections 3–4 (architecture and domain model) as the contract everything else conforms to.

Where we start. From scratch. A demonstration system exists and proves the product — the quote, the journey, the calculator all work and have been shown to people. Treat it as proof of concept only. We are building the real platform new.

> **The one rule before anything else — the language.**
> The product is a **mortgage**, never a "loan." The customer **makes no payments**: no repayments, no interest out of pocket. The mortgage interest is serviced by an **investment account**, not by the homeowner. There is **no repayment schedule, no arrears, no collections**. If you find yourself modelling a repayment or an arrears state, stop — you have misread the product; re-read Section 2.
> Two more: never say "robo-advice" (say **AI-guided advice**); the five agent names are **internal** and a customer never hears them.

1. The shape of the thing

FutureProof owns the product, the intellectual property and the platform. **Licensed lenders run on it — one or several per market** — each white-labelled, under its own licence. We own the engine; the lenders drive.

The system divides cleanly into two planes:

- **Control plane** — owned and run centrally by FutureProof: the product brain (pricing, actuarial models, business rules), the funding layer (Wholesale Funders), the AI agent models, and model/compliance governance. Consumed by every tenant, versioned, never forked.
- **Execution plane** — one isolated, white-labelled tenant per lender: its own branding, domain, data and licence. The tenant runs the operation; it cannot change the product.

A **market** is a jurisdiction (AU, NZ, UK, US): it fixes the regulator, currency, product-config bounds, distribution mode and data-residency region. **Lenders are tenants that operate within a market** — one or several per market.

The build is four workstreams (Sections 5–8), delivered over the roadmap in Section 15:

#	Workstream	One line
A	Multi-tenant, white-label foundation	Many isolated, branded lenders on one codebase
B	The product brain	One central, versioned pricing/rules service every tenant calls

#	Workstream	One line
C	Wholesale Funders (funding layer)	The shared funding stack a mortgage is booked into
D	Agents & the AI gateway	Five agents that run the operation, behind the advice wall

2. The product, in build terms

The product is a **mortgage that pays the homeowner**. A homeowner with substantial equity takes it out, receives a **guaranteed income**, and makes **no payments**. The mortgage's interest, and the income, are paid from an **investment account** funded at origination. The principal is settled at the end of term, from sale of the property or refinancing.

To the customer it is the **Guaranteed Income Plan**; internally and to partners, the **EPM**.

What the build must honour

These are fixed product mechanics, not design choices. Get them wrong and the whole model breaks.

- **No customer payments, ever.** No repayment schedule, no direct debit, no interest billing, no arrears, no collections, no "pay off early" path. The system never asks the customer for money during the life of the mortgage.
- **The investment account services everything.** It is funded at origination, pays the homeowner's income, and services the mortgage interest. Its target allocation is roughly **70% S&P 500 ETFs / 30% fixed income**. Its health — not "is the customer paying" — is the thing the system watches.
- **Guaranteed income** is roughly **1.5% of property value per annum**. The exact figure comes from the pricing model (Workstream B), not from application code.
- **Run-off, not prepayment.** There are no voluntary prepayments or early terminations until the cost of the mortgage has been met from the investment account. This removes early-exit risk by design — model it as a `run_off` state, not as an early-repayment flow.
- **Investment holiday, not arrears.** If investments underperform, income may pause — an **investment holiday** — rather than the customer falling behind. The portfolio metric is **Investment Health** (share of accounts in good standing). The word "arrears" must not appear in schema, code or UI.
- **House price is fixed at origination** for the purpose of setting the mortgage (it sets the LVR). This is not a shared-appreciation product; do not model ongoing house-price movement in the product.

Product parameters (origination)

Parameter	Value	Notes
LTV	up to 80%	Per-market ceiling set in market config
Term	10 / 15 / 20 / 25 / 30 years	
Guaranteed income	~1.5% of property value p.a.	From the pricing model, per term/LTV
Investment allocation	~70% equity ETFs / ~30% fixed income	Target; rebalanced by Yumi
Settlement	end of term, from sale or refinance	Principal + accrued cost

Risk vocabulary (you will see it in the model)

The actuarial model speaks in **PoC** (Probability of Claim — an insurance claim on an expiring mortgage; the headline risk metric) and **PoD** (Probability of Deficit — a point-in-time balance-sheet snapshot). Claims are covered by lenders' mortgage insurance (LMI, ~90% of loss) with a tail-risk reinsurance layer above. You do not need to re-derive these to build the platform — you call the model (Workstream B) and store what it returns.

3. Architecture

3.1 Principles

- **Modular monolith.** One Rails application, internally partitioned into bounded contexts with explicit interfaces. No microservices, no message bus, until scale forces them.
- **Server-side business logic.** All business logic runs on the server. The browser does presentation only (Stimulus/Hotwire). No business rules in JavaScript, no SPA.
- **One source of truth for the product.** Pricing, models and rules live in the control plane and are versioned. Tenants consume; they never copy or fork.
- **Isolation by default.** A request is bound to exactly one tenant for its whole lifecycle. No code path reads one lender's data while serving another.
- **Everything consequential is logged.** Every state change and every agent action is auditable and explainable.

3.2 The stack

Layer	Choice	Why
Framework	Ruby on Rails (8.x)	Token-efficient for AI-built code; convention over configuration; batteries included
Language	Ruby 3.4	
Database	PostgreSQL	Per-jurisdiction clusters; schema-per-lender isolation; native multi-DB in Rails
UI	Server-rendered ERB + Stimulus/Hotwire	Logic stays server-side; no separate frontend stack
Styling	Custom CSS design system	Per-tenant theming; no third-party framework
Background jobs	Solid Queue	Runs on Postgres; no extra infrastructure
Cache	Solid Cache	Same
Hosting	Fly.io	Multi-region; a region per jurisdiction for residency
AI	Claude API (Anthropic)	Agent reasoning; see Section 8

3.3 Module map (bounded contexts)

Organise the monolith into packages (Packwerk-style), each with a public interface and private internals:

```
Tenancy      resolve tenant, switch connection, provision, theming, config
Identity     users, sessions, roles (customer / adviser / staff / agent)
Origination  the application journey: enquiry -> quote -> submit -> assess -> settle
ProductBrain pricing, actuarial model adapter, business rules, product versions
Funding      Wholesale Funders: funder pools, allocations, investment accounts, hedge
Agents       the AI gateway, the five agents, tasks, approvals
Governance   audit log, model & compliance governance, advice-wall enforcement
Notifications email / messaging templates and delivery
Admin        staff tooling and dashboards (per tenant)
```

Dependencies point inward toward ProductBrain and Tenancy; nothing depends on a tenant's web layer. Cross-context calls go through published interfaces only.

3.4 Multi-tenancy mechanism (decision)

Decision: schema-per-lender, inside one PostgreSQL cluster per jurisdiction.

- Each lender (tenant) gets its **own schema** holding all of its business tables. There is no shared `tenant_id` column on business tables — isolation is structural.
- All tenants of a market live in **that market's Postgres cluster**, hosted **in the market's region** — so UK data physically stays in the UK.
- A small **central (shared) schema** holds cross-tenant control-plane data: the tenant registry, market config, product versions, and the central audit index.
- Tenant resolution (Section 5) sets the connection's `search_path` to the tenant's schema for the life of the request.
- **Escape hatch:** a high-volume lender, or a partner whose contract demands physical separation, can be **promoted to its own database** — same code path, different connection.

Why not the alternatives:

- *Shared schema* + `tenant_id` *column* — one query bug leaks across partners; the JV contracts demand walls. Rejected.
- *Database-per-lender for everyone* — strongest isolation but heavy operational cost (migrations, connection pools, backups × N) at the scale we start at. Adopted only via the escape hatch where required.

> **Acceptance:** any business query, run without a resolved tenant, returns nothing (or raises) — never another tenant's rows. This is enforced by the connection layer and proven by tests (Section 11).

3.5 Request flow

```
Customer/Adviser
   |  (HTTPS, tenant domain)
   v
[Tenant resolver]  -- host -> tenant -> set schema + load config/theme
   v
[Tenant web app]   application journey | admin | dashboards
   |            \                 \
   | quote       | agent action    | enqueue job
   v             v                 v
[ProductBrain]  [AI Gateway]      [Solid Queue]
   |  (versioned quote)              |
   v                                 v
[Wholesale Funders]  <-- book mortgage, open investment account
   |
   v
[Governance] audit every consequential step (cross-cutting)
```

3.6 Repo & app layout

```
app/
  domains/            # bounded contexts (Packwerk packages)
    tenancy/  identity/  origination/  product_brain/
    funding/  agents/  governance/  notifications/  admin/
  controllers/        # thin; delegate to domain services
  views/              # ERB, themed per tenant
  javascript/         # Stimulus controllers only
  assets/stylesheets/ # design system + per-tenant theme variables
config/
  markets/            # one file per market (jurisdiction) config
db/
  central/            # migrations for the shared/control schema
  tenant/             # migrations applied to every tenant schema
```

4. Domain model

Entities below are greenfield. Names are indicative; fields list the ones that matter. "Central" = shared schema; "Tenant" = per-lender schema.

4.1 Core entities

Entity	Schema	Key fields	Purpose
Market	Central	code, currency, regulator, ltv_ceiling, distribution_mode, residency_region	A jurisdiction and its rules
Tenant (Lender)	Central	market_code, name, slug, domains[], schema_name, licence_ref, status, theme	A licensed lender; one or several per market
ProductVersion	Central	semver, model_ref, params, constraints, status	A versioned product/pricing config
User	Tenant	role (customer/adviser/staff), email, auth, status	A person in a tenant
Applicant	Tenant	user_id, dob, contact, kyc_status	A homeowner applying
Property	Tenant	address, valuation, value_at_origination	The security
Application	Tenant	applicant_id, property_id, state, quote_id	The journey (state machine, 4.2)
Quote	Tenant	product_version, term, ltv, income_pa, fees, projection, issued_at	Immutable priced offer
Mortgage	Tenant	application_id, principal, term, start/end, state	The EPM contract post-settlement (4.3)
Investment Account	Tenant	mortgage_id, balance, allocation, health, holiday_state	Services income + interest (4.4)
IncomePayment	Tenant	investment_account_id, amount, due_on, paid_on	Income to the homeowner
AgentTask	Tenant	agent, type, subject_ref, state, proposed_action, approval	A unit of agent work (Section 8)
AuditEvent	Tenant + Central	actor, action, subject_ref, before/after, occurred_at	Tamper-evident record

4.2 Application state machine

```
enquiry -> quoted -> submitted -> in_assessment -> approved -> settled
                                        |              |
                                        +-> declined   +-> withdrawn
```

- `enquiry` -> `quoted`: ProductBrain returns a Quote.
- `submitted`: applicant accepts a quote and provides details/documents.
- `in_assessment`: KYC/AML, valuation, eligibility checks (rule-based; Rie/Akane assist).
- `approved` -> `settled`: offer accepted; Funding books the mortgage and opens the investment account; a `Mortgage` is created.

4.3 Mortgage state machine

```
active -> investment_holiday -> active       (income pauses/resumes)
active -> run_off -> settled                 (cost met from investment a/c, then term end)
```

There is **no** `in_arrears`, `delinquent`, `repaying`, or `prepaid` state. If one appears, it is a bug (Section 2).

4.4 Investment account

- Opened and funded at origination by the Wholesale Funders layer.
- Pays `IncomePayments` to the homeowner on schedule; services the mortgage interest internally.

- **Investment Health** is derived (e.g. funded ratio vs. obligations); below a configured threshold the account enters `investment_holiday` (income pauses).
- Rebalanced toward the target allocation by **Yumi** (Section 8).
- At term, contributes to settlement; surplus distribution is a funding-side concern (Section 7), never the customer's.

5. Workstream A — Multi-tenant, white-label foundation

Goal. Many branded lenders — one or several per market — on one codebase, each isolated, each configurable within bounds, with two distribution modes (self-service and adviser-led).

5.1 Tenant resolution

- Middleware maps the incoming **host** to a `Tenant` (central registry), then sets the request's tenant for its whole lifecycle.
- The database connection switches to the tenant's **schema** (`search_path`) — or its dedicated database if promoted.
- The tenant's **config** (market rules + tenant overrides within bounds) and **theme** are loaded into the request context.
- Background jobs carry the tenant identity explicitly and re-establish the same binding when they run.

```
# illustrative
Current.tenant = TenantRegistry.resolve!(request.host)   # raises if unknown host
TenantConnection.with(Current.tenant) do                  # sets schema / database
  Current.config = MarketConfig.for(Current.tenant)       # market + tenant overrides
  yield
end
```

5.2 Configuration, not forks

- **Market config** (per jurisdiction, in `config/markets/<code>.yml`): currency, LTV ceiling, regulator, distribution mode, residency region, allowed product versions.
- **Tenant overrides** (per lender): branding, domains, language, contact details, and any settings the market marks as tenant-tunable — all **validated against FP-governed bounds** at write time. A tenant cannot set a value outside its market's bounds.
- Pricing, the actuarial model and the business rules are **not** configurable here — they live in the product brain (Workstream B).

5.3 White-label / theming

- One design system; per-tenant theme = a set of brand tokens (colour, logo, type, copy) applied via CSS custom properties. No per-tenant CSS forks.
- Per-tenant domains, email templates, language. One codebase renders all tenants.

5.4 Distribution modes

- **Self-service** (AU/NZ pattern): the customer drives the journey directly.
- **Adviser-led** (UK pattern): a licensed adviser drives the journey on the client's behalf; the customer-facing self-service journey cannot represent this — it is a distinct flow (Section 9). The mode

is set in market config.

5.5 Provisioning a new tenant

A repeatable operation (a script / Motoko task):

1. Create the `Tenant` record in the central registry (market, slug, domains, licence).
2. Create its schema (or database) and run all tenant migrations against it.
3. Seed reference data and the tenant theme/config.
4. Wire domains and TLS.
5. Smoke-test: resolve the host, render the journey, request a quote.

5.6 Migrations across tenants

- Tenant migrations live in `db/tenant/` and must run against **every** tenant schema (and any promoted databases). A migration runner iterates the registry.
- Migrations are reversible and idempotent (Section 12). A migration that cannot run cleanly against all tenants does not ship.

> **Acceptance (A).** A new lender can be stood up by running the provisioning operation — its data is in its own schema/database, its brand renders, it can produce a quote — with no code fork. A cross-tenant query without a resolved tenant returns nothing. Two lenders can coexist in one market, fully isolated.

6. Workstream B — The product brain

Goal. One central, versioned service that answers "what is the quote and the terms for this customer?" Every tenant calls it; none can edit it.

6.1 Boundary

- A bounded context (`ProductBrain`) with a **published interface**, callable in-process now and extractable to a service later without changing callers.
- Inputs come from the tenant; the answer comes from **our engine and our current models**. A tenant never receives a copy of the model to run, tweak or leak.

6.2 The quote interface

```
ProductBrain.quote(
  market:,            # jurisdiction code
  product_version:,   # semver; defaults to market's current
  property_value:,    # at origination
  ltv:,               # <= market ceiling
  term_years:,        # one of the allowed terms
  applicant:          # age etc. as the model requires
) -> Quote {
  income_pa:,         # guaranteed income p.a.
  fees:,
  investment_plan: { equity_pct:, fixed_income_pct:, initial_funding: },
  projection:,        # balances over term (for display)
  risk: { poc:, pod: },
  product_version:, issued_at:
}
```

- A `Quote` is **immutable and versioned** — it records the exact `product_version` used, so it is reproducible.
- Validation rejects inputs outside market bounds (LTV ceiling, allowed terms) before the model runs.

6.3 The actuarial model adapter

- The engine implements the **validated EPM model** (the demonstrated Tom lookup and Pavel Monte-Carlo models). This document does not restate the model maths — it is owned in the model spec and wrapped behind a stable adapter interface.
- The adapter takes the inputs above and returns income, fees, investment plan, projection and risk metrics (PoC/PoD).
- **Hard constraints** (the model's C1–C6 and parameter bounds) are enforced in the adapter; a request that would violate them is rejected, not silently clamped.

6.4 Versioning & governance

- Every change to pricing, model or rules is a new `ProductVersion` (semver), reviewed under model governance (Section 14) before activation.
- Markets pin an **allowed set** of product versions; the "current" version is what new quotes use. Old quotes remain reproducible against their pinned version.
- Changing a rule changes it in one place and takes effect for all tenants on that version at once.

> **Acceptance (B).** A tenant can obtain a quote and terms only by calling the central service; the same inputs + product version always reproduce the same quote; an out-of-bounds input is rejected; activating a new product version changes new quotes everywhere on that version with no tenant code change.

7. Workstream C — Wholesale Funders (funding layer)

Goal. The shared funding stack behind every mortgage. Built once, centrally; each lender plugs in and books a mortgage to funding without its own capital-markets desk.

7.1 Entities

Entity	Key fields	Purpose
Funder	name, type, capacity, terms	A wholesale funding source
FunderPool	funder_id, balance, allocation_rules	A pool capital is drawn from
FundingAllocation	mortgage_ref, pool_id, amount, booked_at	Links a mortgage to funding
InvestmentAccount	mortgage_ref, balance, allocation, health	Services income + interest (Section 4.4)
InsurancePolicy	mortgage_ref, insurer, coverage_pct	LMI (~90% of loss)
ReinsuranceLayer	attaches_at, limit, reinsurer	Tail-risk layer above LMI
HedgePosition	instrument, notional, as_of	Portfolio-level S&P 500 hedge

7.2 Booking flow (at settlement)

1. `Funding.book_mortgage(mortgage)` -> selects a pool by allocation rules -> creates a `FundingAllocation`.

2. `Funding.open_investment_account(mortgage)` -> funds it from the booking -> sets target allocation (70/30).
3. Attach `InsurancePolicy` (LMI) and assign to the relevant `ReinsuranceLayer`.
4. Register the mortgage's exposure with the portfolio `HedgePosition` management.

7.3 Investment account lifecycle (funding side)

- Pays scheduled income and services interest from the account.
- **Run-off mechanism:** no exit until the mortgage cost is met from the account.
- **Investment holiday:** when Investment Health falls below threshold, income pauses (the account, not the customer, is the subject).
- At term: settle from property sale/refinance; **surplus is split 50/50 between FutureProof and the mortgage funder — never the borrower.**

7.4 Interfaces

```
Funding.book_mortgage(mortgage)        -> FundingAllocation
Funding.open_investment_account(m)     -> InvestmentAccount
Funding.record_income_payment(account) -> IncomePayment
Funding.investment_health(account)     -> Health
Funding.settle(mortgage)               -> Settlement
```

Acceptance (C). A settled mortgage is booked to a funder pool, has an investment account opened and funded with the target allocation, and carries LMI + reinsurance — all through the Wholesale Funders layer, with no per-lender funding desk. Income payments and holidays operate on the account; surplus at term splits 50/50 FP/funder.

8. Workstream D — Agents & the AI gateway

Goal. Five agents run the operation in every market at once, behind one controlled, audited path, and behind the advice wall. A human holds every consequential decision until a step is earned.

This section is the summary. The full build — runtime (Claude API + manual agentic loop), the gateway, per-agent specs, the human-in-the-loop / approval-queue model, guardrails, evals and the model tiering — is the dedicated **AI Architecture & Build Spec** (`AI_BUILD_SPEC.md`). Build the agent layer from that; this is the orientation.

8.1 The five agents

Agent	Domain	Example work
Akane	Acquisition	First contact, qualification, guiding the application
Misato	Service	Customer comms and service after onboarding
Rie	Back office	Document checks, assessment support, operations
Yumi	Investment account	Monitoring health, rebalancing toward target
Motoko	Engineering / ops	Builds and runs the other four; platform tasks

Internal names only — never shown to customers.

8.2 The AI gateway

Every agent action takes one controlled path:

```
trigger (event or schedule)
  -> assemble context  (tenant-scoped data only)
  -> model call        (Claude API; tools = published domain interfaces)
  -> proposed action   (an AgentTask)
  -> capability check   (advice-wall + step level, 8.4)
  -> human approval     (per step, 8.3) OR auto-execute
  -> execute via domain interface
  -> audit (8.5)
```

- Agents act **only** through published domain interfaces — never raw SQL, never cross-tenant.
- Context assembly is tenant-scoped; one market's data never enters another's prompt.

8.3 The staircase (capability levels)

Step	Agents do	Control
0 — today	Timed messages, lifecycle stages, handoffs (rule-based)	The rules
1 — draft	Draft the reply, the assessment, the summary	Human approves and sends
2 — decide	Handle clear-cut cases; flag the rest	Human on exceptions
3 — anticipate	Spot the stuck application / at-risk account, and act	Human sets guardrails

Capability is assigned **per agent, per action type** — an agent may be at step 2 for one action and step 1 for another. Start every action at step 1.

8.4 The advice wall (enforced, not just stated)

A hard line separates **general guidance** (allowed) from **personal financial advice** (telling a specific customer this product suits *them* and they should take it — an AFSL + Statement of Advice in AU; human-adviser-only in the UK).

Enforcement:

- Agent action types are an allow-list; "recommend the product to this customer" is not on it.
- Outputs are checked for advice-like content before send; anything near the line routes to a human.
- In the UK, customer-facing recommendation is reserved to the human adviser; agents support, never replace.
- Two invariants everywhere: models are ours and never leak one lender's data into another's; every real decision is logged and explainable.

8.5 Audit

Every proposed action, approval/rejection, and execution writes an `AuditEvent` (actor, action, subject, before/after, the model version and prompt reference). The trail is sufficient to explain *why* any agent did anything.

Acceptance (D). Each agent operates at its assigned step with a human in the right place; an agent can only act through published interfaces and only within its tenant; advice-like output is blocked or routed to a human; every action is logged and explainable.

9. Customer & adviser journeys

9.1 Self-service (AU / NZ)

1. **Land** — branded entry for the tenant.
2. **Eligibility & quote** — property value, equity, term; calls ProductBrain; shows the guaranteed income, the plan, and a clear projection. Plain language; never the language of debt.
3. **Apply** — applicant details, property, documents.
4. **Verify** — identity / KYC / AML.
5. **Assess** — eligibility and valuation (rule-based; Akane/Rie assist; human on exceptions).
6. **Offer & accept** — present terms; capture acceptance.
7. **Settle** — Funding books the mortgage and opens the investment account; a `Mortgage` is created.
8. **Live** — dashboard shows income payments and **Investment Health** (never "balance owing"); Misato handles service.

9.2 Adviser-led (UK)

- An **adviser portal**: the adviser sets up the client, conducts **suitability** (the regulated human step), then drives quote -> application on the client's behalf.
- The customer-facing self-service journey is not used to recommend; the agent layer supports the adviser, it does not advise the client.

10. How we build — the dev loop

We build the modular monolith component by component, a small team alongside AI coding agents. For each component:

1. **Refresh the prompt** — load the layered build context from `docs/prompts/`: `master.md` (the shared core) + the relevant `constituents/<domain>.md` (and `agents/<agent>.md` if building an agent). Pin the inputs/outputs, the definition of done, and the rules (terminology, isolation, CSP). The non-negotiables live only in `master.md` — don't restate them.
2. **Collect test data** — realistic fixtures (customers, properties, quotes, market configs) that become what the tests run on.
3. **Build** — the smallest slice that meets the acceptance criteria. Business logic server-side; Stimulus for UI only; calls go through domain interfaces.
4. **Test** — integration-first, then the real path in a browser (Section 11).
5. **Review** — diff against correctness and the guardrails (terminology, tenant isolation, CSP, no over-engineering); run the quality gate.
6. **Deploy** — ship the slice behind config; verify in the running app (Section 12). Then the next component.

Repo conventions: feature branch per slice; PR into **master** (the trunk); CI green before merge; one logical change per PR.

10.1 Change control — one door, graded locks, earned keys

The loop above is *how a change is made*; this is *how any change gets in*. Four actors author changes — the CTO, business users (via the admin's prompt/change-request bridge, never touching git), the local AI coding agent, and the Claude GitHub Action (plain-language change requests -> draft PRs). All four converge on one door: a pull request into master carrying a recorded impact assessment, the required CI check, and the CTO's review — merge is the only path to release, and no agent merges. Changes face locks graded by **risk tier** (prompt wording -> general code -> financial core/auth/migrations -> deploy & credentials), and agents earn per-tier autonomy on a measured agreement rate, on the same capability staircase that governs the runtime agents (AI spec §6). The canonical statement — actors, door, tiers, staircase, incident path — lives in `docs/OPERATING_MODEL.md`; it is referenced here and not restated.

11. Testing strategy

- **Layers.** Unit (domain logic), integration (a request through a context), system (the real journey in a browser). The seven-step protocol in `CLAUDE.md` is the standard: write the test, run it, hit the real URL, verify the render, exercise interactions, run the full suite, then claim done.
- **Tenant isolation tests are mandatory.** Prove that a query without a resolved tenant returns nothing, and that tenant A can never read tenant B's data. This is the highest-priority test class.
- **Test data.** A fixtures strategy that stands up a market, two tenants, and representative applicants/properties/quotes; the data collected in step 2 of the loop feeds these.
- **Model conformance.** ProductBrain has golden-master tests: fixed inputs + product version -> exact expected quote (reproducibility).
- **CI gates on every change:** full suite, security scan (Brakeman), style (RuboCop), CSP check. Nothing ships on a red suite — the test job is a required branch-protection check on master.
- **Prompt changes get their own gate:** golden-transcript evals plus advice-wall red-team checks run in CI whenever `docs/prompts/**` changes, so a prompt PR arrives with evidence, not just a diff (AI spec §8).

12. Deployment & environments

- **Hosting:** Fly.io, with a **region per jurisdiction** so each market's data and compute sit in-region (residency).
- **Environments:** merge to master deploys to **staging**; production is a deliberate, manual promotion by the CTO after using the staging site. "Test, then deploy" is structural, not a discipline. Migrations run as the release command, so schema and code ship together and a failing migration aborts the release.
- **Pre-flight (fixed):** confirm the account, confirm the app, confirm status — then deploy (remote-only).
- **Migrations:** tenant migrations run against every tenant schema (and promoted databases); reversible and idempotent; a migration that cannot run cleanly against all tenants does not ship. Use the safe-migration pattern (forward-compatible, backfill in batches, no destructive drops without an explicit, separate, approved step).

- **Release behind configuration, per lender and per market.** A lender or market can be switched on, or a feature rolled out, without touching another.
- **Data safety is absolute:** never drop, reset, truncate or overwrite data without explicit permission.

13. Operations & management

- **Run by the agents, governed by people.** Operations run on the five agents, with a person on every consequential decision; Motoko builds and runs the other four.
- **The signal is Investment Health** — the share of accounts in good standing — per tenant and per portfolio. There is no "arrears" signal because there are no customer payments.
- **System health:** standard app/DB/queue monitoring, per region.
- **Audit & governance:** every real decision logged and explainable; model and compliance governance owned centrally; the advice wall enforced and monitored.
- **Backups & recovery:** per-jurisdiction backups; restore tested; tenant-level restore possible because tenants are schema/DB-isolated.
- **Incident:** rollback via reversible deploys; tenant isolation limits blast radius to one lender.

14. Security, compliance & data residency

- **Tenant isolation** is structural (schema/DB per lender) and enforced at the connection layer; proven by tests.
- **Data residency:** each market's data lives in its jurisdiction's cluster/region. Central control-plane data (tenant registry, product versions, model config) holds **no customer PII**; where a central service touches customer data, it does so in-region.
- **Encryption:** at rest (database, backups) and in transit; application secrets in encrypted credentials; PII fields encrypted where required.
- **Regulatory boundaries:** the advice wall (Section 8.4); AU AFSL/Statement-of-Advice limits; UK adviser-led requirement; AML/CFT in the journey; record-keeping via the audit log.
- **Secure by default:** Rails CSRF/SQLi/XSS protections; strict CSP (no inline styles/scripts/handlers); least-privilege roles for staff and agents.
- **No certification theatre:** SOC 2 / ISO 27001 are earned with UK/US operations, not claimed before.

15. Phased roadmap

Spine first, then a thin end-to-end slice, then deepen by constituent. Dependency-ordered, market-agnostic (Australia is the live build; a second lender/market proves the platform). The five constituents — **Wholesale Funders · Lenders · Brokers · Investments · Customers** — sit on a platform spine (product brain, agents, governance). Per-domain build prompts live in `docs/prompts/`.

Phase 0 — Spine

The foundation everything hangs off: repo + CI (suite + Brakeman + RuboCop + CSP), the multi-tenant **Lenders** foundation (schema-per-lender, tenant resolver, config, theming, provisioning), the central **product brain** (versioned quote service), identity, and **governance** (audit log).

- **Done when:** one lender tenant (AU) stands up by config and is isolated; a customer can get a reproducible quote from the central engine; every consequential action is logged. (Acceptance A + B.)

Phase 1 — Walking skeleton (thin, end-to-end)

One thin slice through the load-bearing constituents, so there is something to test and show — one funder stub -> one lender -> one customer -> one quote -> one mortgage -> one funded investment account:

- **Wholesale Funders (stub):** book a settled mortgage to a pool.
- **Customers (journey):** enquiry -> settled for one lender, creating a `Mortgage`.
- **Investments:** open + fund the account, pay one income payment, report Investment Health.
- **Brokers:** attribution field only — do not build the channel yet.
- **Done when:** an application runs enquiry -> settled end-to-end, producing a funded mortgage + investment account, on the platform spine.

Phase 2 — Agents, step 1 (the AI layer)

The gateway, the manual loop, the approval queue, the guardrail/advice-wall checks; the five agents at **step 1** (draft, human approves) on their first action types.

- **Done when:** Acceptance (D) at step 1 — an agent proposes, a human approves/edits/rejects, it executes only on approval, all logged and tenant-isolated. (See `AI_BUILD_SPEC.md`.)

Phase 3 — Deepen by constituent

With the skeleton running and testable, deepen each domain as we learn:

- **Lenders:** adviser-led mode; a second lender/market (the platform proof — no product rebuild).
- **Wholesale Funders:** insurance/reinsurance, the S&P 500 hedge, capital rules, surplus split.
- **Investments:** rebalancing, holiday logic, settlement at term.
- **Customers:** richer assessment, documents, the live servicing dashboard.
- **Brokers:** the broker channel (portal, accreditation, commissions) when go-to-market needs it.
- **Agents:** steps 2–3 per action type as evals earn them.
- **Done when:** a second lender/market is live on config alone; each constituent is at production depth; agents handle clear-cut cases with humans on exceptions.

> **The milestone that proves the platform** is still the **second lender/market on config, with no product rebuild** — reached in Phase 3 once the skeleton is real. Until then, the engine is not yet an engine.

16. Risks & open questions

- **EPM model source of truth.** The pricing/actuarial maths is owned in the model spec, not here; the adapter (6.3) must wrap the validated engine exactly. *Open:* which engine artefact is canonical for the build, and its exact I/O contract.
- **Central services vs residency.** ProductBrain and the agent models are central; confirm they hold no customer PII, or run them in-region per market where they do.
- **LLM reliability in a regulated flow.** Step progression (8.3) must be evidence-led; define the evals and error budgets that earn each step.
- **Tenant migration at scale.** Schema-per-lender migrations are O(tenants); define the runner's batching, timeout and partial-failure handling before tenant count grows.
- **Promotion to dedicated DB.** Define the trigger and the (online) migration path before the first promotion.
- **AML/KYC providers** per jurisdiction — to be selected.

17. Glossary

Term	Meaning
EPM / Guaranteed Income Plan	The product: a mortgage that pays the homeowner a guaranteed income; the customer makes no payments
Investment account	The ~70/30 account funded at origination that pays the income and services the mortgage interest
Investment holiday / Investment Health	Income pause on underperformance; the share of accounts in good standing. The EPM has no "arrears"
Run-off	No voluntary prepayment/early exit until the mortgage cost is met from the investment account
Control plane	The product brain, funding, agent models and governance FutureProof owns and runs
Execution plane	The licensed lenders (one or several per market), white-labelled, on the platform under their own licence
Market	A jurisdiction (AU/NZ/UK/US): regulator, currency, config bounds, residency region, distribution mode
Tenant / Lender	A lender's isolated instance: its own schema (or database), branding, domain and config
Wholesale Funders	The shared funding layer (funder pools, insurance, reinsurance, S&P 500 hedge, investment accounts)
Product brain	The central, versioned pricing/actuarial/rules service every tenant calls and none can edit
The advice wall	The hard line between general guidance (allowed) and personal financial advice (licensed; UK human-adviser-only)
The five agents	Akane (acquisition), Misato (service), Rie (back office), Yumi (investment account), Motoko (eng/ops). Internal names
PoC / PoD	Probability of Claim (headline risk) / Probability of Deficit (point-in-time snapshot)

Internal — FutureProof. Full technical build specification; greenfield design. Build from this; keep it current as components land.